



Microservices Training - Trainer Playbook

Payment Domain Implementation - 4-Day Course



PRE-SESSION SETUP (Do this 30 min before each day)

Prerequisites Check

```
# Verify Java 17
java -version
# Expected: openjdk version "17.x.x"

# Verify Maven
mvn -version
# Expected: Apache Maven 3.9.x

# Verify Docker Desktop is running
docker info
# Expected: Server Version: xx.x.x

# Clone/navigate to project
cd microservices-training

# Build all modules
mvn clean install -DskipTests
# Expected: BUILD SUCCESS for all 6 modules

# Start infrastructure only (fast startup for first demo)
docker-compose up -d redis kafka zookeeper zipkin prometheus grafana kafka-ui redis-
commander nginx-cdn

# Wait for kafka to be ready (30-60 seconds)
docker-compose logs -f kafka | grep "started"
```

Verify Infrastructure Running

```
# Redis
docker exec payments-redis redis-cli ping
# Expected: PONG

# Kafka
docker exec payments-kafka kafka-broker-api-versions --bootstrap-server localhost:9092
# Expected: lists broker versions

# Zipkin
curl http://localhost:9411/health
# Expected: {"status":"UP"}
```

DAY 1 PLAYBOOK

Theme: API Design + OpenAPI + Microservice Structure + Resiliency Basics

MODULE 1.1: Payments API Recap (1 hour)

Learning Objectives

- Understand REST resource modeling for payments
- Know tradeoffs between cursor vs offset pagination
- Understand URL versioning and backward compatibility

Setup (5 min)

```
# Start payment-service only
cd payment-service
mvn spring-boot:run
# Wait for: "Tomcat started on port(s): 8081"
```

Demo 1: REST Resource Modeling (15 min)

Show and explain the URL structure:

```
POST   /api/v1/payments      # Create (not /api/v1/createPayment!)
GET    /api/v1/payments/{id} # Read one
GET    /api/v1/payments     # Read many (with filters)
DELETE /api/v1/payments/{id} # Cancel
POST   /api/v2/payments/{id}/refund # Sub-resource action
```

Trainer talking points:

- Resources are NOUNS, not verbs (/payments not /makePayment)
- HTTP verbs ARE the verbs (POST=create, GET=read, DELETE=cancel)
- Sub-resource action /refund is acceptable for complex operations
- Plural noun for collections: /payments not /payment

Live demo:

```
# Create first payment
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: demo-001" \
  -d '{
    "orderId": "ORDER-001",
    "customerId": "CUST-001",
    "amount": "9.99",
    "currency": "USD",
    "paymentMethod": "CARD",
    "cardNumber": "4111111111111111",
    "idempotencyKey": "demo-001"
  }'
```

```
}'
```

```
# Point out: 201 Created, X-Payment-ID header, masked card number
```

Demo 2: Offset vs Cursor Pagination (20 min)

Offset Pagination (v1):

```
# Page 0, size 5
curl "http://localhost:8081/api/v1/payments?customerId=CUST-001&page=0&size=5"
# Show: page, size, totalElements, totalPages

# Page 1
curl "http://localhost:8081/api/v1/payments?customerId=CUST-001&page=1&size=5"

# With filter
curl "http://localhost:8081/api/v1/payments?customerId=CUST-001&status=COMPLETED"
```

Cursor Pagination (v2):

```
# First page (no cursor)
curl "http://localhost:8081/api/v2/payments?customerId=CUST-001&size=5"
# Show: nextCursor (Base64), hasMore=true

# Decode cursor to show what's inside
echo "BASE64_CURSOR_HERE" | base64 -d
# Shows: 2024-01-15T10:30:00Z|payment-id-xyz

# Next page using cursor
curl "http://localhost:8081/api/v2/payments?customerId=CUST-001&size=5&cursor=BASE64_CURSOR_HERE"
```

Key Discussion Points:

	Offset	Cursor
SQL	LIMIT x OFFSET y	WHERE id < lastId
Page drift	YES (inserts shift pages)	NO (anchored to record)
Random access	YES (jump to page 5)	NO (must traverse)
COUNT query	YES (expensive)	NO
Use case	Admin reports	Real-time feeds

Demo 3: API Versioning (15 min)

Show both versions side by side:

```
# V1 response - no pspReference
curl http://localhost:8081/api/v1/payments/PAYMENT-ID
```

```
# V2 response - includes pspReference
curl http://localhost:8081/api/v2/payments/PAYMENT-ID

# Compare: V2 adds pspReference - backward compatible addition
```

Show backward compatibility rules:

```
# V1 still works - never broken
curl http://localhost:8081/api/v1/payments/PAYMENT-ID
# V1 clients never see pspReference - not breaking them
```

Discussion: Breaking vs Non-Breaking Changes

-  Safe: Add new optional field (V2 adds pspReference)
-  Safe: Add new endpoint
-  Breaking: Remove field from V1
-  Breaking: Change field type
-  Breaking: Make optional field required

Demo 4: Idempotency (10 min)

```
# First request
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: SAME-KEY-001" \
  -d '{"orderId":"ORDER-001","customerId":"CUST-001","amount":"9.99","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","jKEY-001"}'
```

```
# SAME request, same key - simulates client retry
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: SAME-KEY-001" \
  -d '{"orderId":"ORDER-001","customerId":"CUST-001","amount":"9.99","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","jKEY-001"}'
```

```
# Point out: SAME paymentId returned - NOT charged twice!
```

Trainer talking point: "Without idempotency keys, a network timeout between PSP charge and HTTP response would mean the client retries and the customer is double-charged. This has caused real financial scandals."

MODULE 1.2: OpenAPI Contract Design (1 hour)

Setup (2 min)

```
# Payment-service should already be running
# Open browser
start http://localhost:8081/swagger-ui.html
```

Demo 1: Swagger UI Walkthrough (20 min)

Walk through each section:

1. Open <http://localhost:8081/swagger-ui.html>
2. Show API metadata (title, version, description, contact)
3. Show server list (local + via gateway)
4. Show Payments V1 group vs Payments V2 group
5. Click "POST /api/v1/payments"
6. Show request schema with validations
7. Show response schemas (201, 400, 422, 429, 503)
8. Show error model (errorCode, message, traceId, fieldErrors)

Demo 2: Try It - OpenAPI Request/Response (20 min)

```
# Get raw OpenAPI spec
curl http://localhost:8081/v3/api-docs | python -m json.tool

# Show it's a complete contract:
# - paths (all endpoints)
# - components/schemas (all models)
# - info (metadata)
```

Execute in Swagger UI:

1. Click "Authorize" -> Enter X-API-Key: demo-key
2. POST /api/v1/payments -> Try It Out -> Execute
3. Show 201 response, response headers
4. POST with invalid data -> Show 400 with fieldErrors
5. POST with amount 75 -> Show 422 (insufficient funds from PSP simulation)

Demo 3: Contract-First vs Code-First (20 min)

Show OpenAPI Config:

```
// Open OpenApiConfig.java
// Show: Info, Servers, SecurityScheme configuration
// Explain: This is CODE-FIRST approach
// Code generates spec automatically via springdoc
```

Show controller annotations:

```
// Open PaymentControllerV1.java
// Point out:
@Operation(summary = "...", description = "...")
@ApiResponses({@ApiResponse(responseCode = "201", ...)})
@Parameter(description = "...")
```

Discussion:

- Code-first: Always in sync, less upfront design
- Contract-first: Design API before coding, better for partners/mobile teams

- Recommendation: Contract-first for external APIs, code-first for internal

🕒 MODULE 1.3: Microservice Structure (1 hour)

Demo 1: Multi-Module Build (20 min)

Show project structure:

```
# Show the full module tree
find . -name "pom.xml" -not -path "*/target/*" | sort

# Build all modules
mvn clean install -DskipTests

# Show build order (Maven respects dependencies)
# payment-common builds FIRST (no internal dependencies)
# payment-service builds AFTER (depends on payment-common)
```

Open parent pom.xml:

```
<!-- Show: modules list, dependency management, shared versions -->
<!-- Explain: payment-common version managed in ONE place -->
<!-- Explain: No version in child poms - inherits from parent -->
```

Demo 2: Dependency Isolation (20 min)

Show the layering:

```
payment-common  <- no internal dependencies (library)
    ↑
payment-service <- depends on payment-common only
order-service   <- depends on payment-common only
bff-service     <- depends on payment-common only
```

Cyclic dependency demonstration:

```
// Open payment-service/pom.xml
// payment-service DOES NOT depend on order-service
// order-service DOES NOT depend on payment-service
// They communicate via Kafka events (loose coupling!)
```

Show shared models:

```
// Open Payment.java in payment-common
// Open PaymentEvent.java in payment-common
// Explain: ONE definition used by ALL services
// Alternative: Each service has own model + mapping (more isolation)
```

Demo 3: Layering Architecture (20 min)

Walk through package structure:

```

payment-service/
├─ controller/      # HTTP boundary (ControllerV1, ControllerV2)
├─ service/         # Business logic (PaymentService)
├─ repository/      # Data access (PaymentRepository)
├─ entity/          # DB persistence (PaymentEntity)
├─ mapper/          # DTO<->Entity conversion (PaymentMapper)
├─ psp/             # External integration (PspClient)
├─ config/          # Configuration (CacheConfig, ResilienceConfig)
├─ filter/          # Cross-cutting (RateLimitFilter)
└─ observability/   # Metrics/Tracing (PaymentMetrics, TraceFilter)

```

Show data flow through layers:

```

HTTP Request
  ↓
PaymentControllerV1.createPayment()
  ↓
PaymentService.createPayment()      (business rules)
  ↓
IdempotencyService.check()          (Redis cache check)
  ↓
PaymentRepository.save()             (H2 persistence)
  ↓
PspClient.chargePayment()            (external PSP call)
  ↓
KafkaTemplate.send()                 (event publishing)
  ↓
PaymentResponse returned up chain

```

MODULE 1.4: Retry & Idempotency (1 hour)

Demo 1: Retry Behavior (20 min)

Trigger retry scenario (amount \$20-\$49.99):

```

# Amount between 10-50 = 30% transient failure rate (triggers retry)
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: retry-demo-001" \
  -d '{"orderId":"ORDER-002","customerId":"CUST-001","amount":"25.00","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","demo-001"}'

# Watch payment-service logs in real time:
# [PSP] Attempting charge #1...
# [PSP MOCK] Simulating transient failure
# [PSP] Attempting charge #2... (retry after 1s)
# [PSP] Charge successful. PSP Ref: PSP-ABC123

```

Show exponential backoff in Resilience4j config:

```
# application.yml
retry:
  instances:
    psp-retry:
      max-attempts: 3
      wait-duration: 1s
      enable-exponential-backoff: true
      exponential-backoff-multiplier: 2.0
      # Attempt 1: immediate
      # Attempt 2: wait 1s
      # Attempt 3: wait 2s
```

Demo 2: Fixed vs Exponential Backoff (15 min)

Draw on whiteboard:

```
FIXED BACKOFF (bad for thundering herd):
Service A fails at t=0
All 1000 clients retry at t=5s -> thundering herd!
Service A fails again -> all 1000 retry at t=10s -> still overwhelmed

EXPONENTIAL BACKOFF (good):
Attempt 1: fail -> wait 1s
Attempt 2: fail -> wait 2s
Attempt 3: fail -> wait 4s
Adds JITTER: each client waits 1s + random(0-500ms)
No synchronized retry storms!
```

Demo 3: Idempotency at PSP Level (15 min)

Show PspChargeRequest has idempotencyKey:

```
// PspChargeRequest.java
private String idempotencyKey; // PSP also supports idempotency!
```

Discussion: "Even the PSP (Stripe, PayPal) supports idempotency keys. If we retry after a timeout, we send the SAME key. PSP returns the SAME charge result without double-charging. Defense in depth: Our service + PSP both prevent duplicates."

DAY 2 PLAYBOOK

Theme: Circuit Breaker, Rate Limiting, Caching, Gateway & Discovery

MODULE 2.1: Circuit Breaker (1 hour)

Setup (5 min)

```
# Ensure payment-service is running
# Open two terminals:
# Terminal 1: watch payment-service logs
```



```
cd payment-service && mvn spring-boot:run
```

```
# Terminal 2: watch circuit breaker status
```

```
watch -n 1 "curl -s http://localhost:8081/actuator/circuitbreakers | python -m json.tool"
```

Demo 1: Circuit Breaker States (30 min)

Step 1: Show CLOSED state (normal)

```
# Successful payment (amount < 10)
for i in {1..5}; do
  curl -X POST http://localhost:8081/api/v1/payments \
    -H "Content-Type: application/json" \
    -H "X-Idempotency-Key: cb-demo-$i" \
    -d '{"orderId":"ORDER-$i","customerId":"CUST-001","amount":"5.00","currency":"USD","paymentMethod":"CARD","cardNumber":"4111demo-$i"}'
  echo ""
done
# Watch CB status: state=CLOSED, failureRate=0%
```

Step 2: Trigger failures (amount >= 100 = slow calls)

```
# Slow calls trigger slowCallRateThreshold (80%)
for i in {1..10}; do
  curl -X POST http://localhost:8081/api/v1/payments \
    -H "Content-Type: application/json" \
    -H "X-Idempotency-Key: cb-slow-$i" \
    -d '{"orderId":"ORDER-SLOW-$i","customerId":"CUST-001","amount":"150.00","currency":"USD","paymentMethod":"CARD","cardNumber":"4111slow-$i"}' &
done
wait
# Watch CB: slowCallRate rising -> eventually state=OPEN
```

Step 3: Show OPEN state (fail-fast)

```
# New payment should IMMEDIATELY fail (no PSP call!)
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: cb-open-test" \
  -d '{"orderId":"ORDER-OPEN","customerId":"CUST-001","amount":"5.00","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","idempotencyKey":"open-test"}'

# Watch logs: "PSP FALLBACK - Circuit breaker OPEN"
# Note: No delay - immediate fallback response!
```

Step 4: Wait for HALF-OPEN (10 seconds)

```

sleep 12
# Watch CB status: state=HALF_OPEN
# Now 3 test calls allowed through

# Send test payment
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: cb-halfopen-1" \
  -d '{"orderId":"ORDER-H0","customerId":"CUST-001","amount":"5.00","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","halfopen-1"}'

# If success: CB transitions back to CLOSED
# Watch CB status: state=CLOSED

```

Key Trainer Points:

- "OPEN state SAVES the PSP from additional load while recovering"
- "Fail-FAST is better than wait-and-timeout when service is down"
- "Half-open is the 'testing the waters' phase"
- "Cascading failure: Payment down -> gateway times out -> all requests slow"

Demo 2: Bulkhead Pattern (20 min)

```

// Show BulkheadConfig.java
// max-concurrent-calls: 10
// Explain: Even if PSP is slow (not failing), bulkhead prevents
// ALL threads from being consumed waiting for PSP

```

```

# Show actuator metrics for bulkhead
curl http://localhost:8081/actuator/metrics/resilience4j.bulkhead.available.concurrent.calls

```

Discussion: "A ship has watertight compartments. If one floods, the ship doesn't sink. Bulkhead isolates the PSP call compartment from the rest of the application."

MODULE 2.2: Rate Limiting (1 hour)

Demo 1: Token Bucket in Action (25 min)

Show rate limit headers:

```

# Normal request
curl -v http://localhost:8081/api/v1/payments?customerId=CUST-001 \
  -H "X-API-Key: demo-key" 2>&1 | grep -E "X-RateLimit|< HTTP"

# See: X-RateLimit-Limit: 60
#       X-RateLimit-Remaining: 59
#       X-RateLimit-Window: 60s

```

Trigger rate limit:

```
# Hammer the API (61 requests rapidly)
for i in {1..65}; do
  status=$(curl -s -o /dev/null -w "%{http_code}" \
    -H "X-API-Key: demo-key" \
    http://localhost:8081/api/v1/payments?customerId=CUST-001)
  echo "Request $i: $status"
done
# Watch: 200s then 429s appear
# After ~61 requests: 429 Too Many Requests
# Header: Retry-After: 60
```

Show per-client isolation:

```
# Client 1 is rate limited
curl -H "X-API-Key: limited-key" http://localhost:8081/api/v1/payments?customerId=C1
# 429 (rate limited)

# Client 2 is NOT affected (separate bucket)
curl -H "X-API-Key: other-key" http://localhost:8081/api/v1/payments?customerId=C2
# 200 (different client, different bucket)
```

Demo 2: Token Bucket vs Quota Discussion (20 min)

Draw on whiteboard:

```
TOKEN BUCKET (rate limiting):
Max: 60 tokens, refill: 1/second

t=0s: [●●●●●●●●●●] 10 tokens (burst)
      Client sends 10 requests -> [          ] 0 tokens
t=1s: [●          ] 1 token (refill)
      1 more request allowed

QUOTA (billing period limit):
Max: 10,000 requests per day
Used: 9,999 -> 1 request left
Tomorrow: resets to 10,000
```

Discussion:

- Rate limiting: Prevents burst abuse, DDoS protection
- Quota: Business model limit (free tier: 1000/day, paid: unlimited)
- Use BOTH: rate limit (per second) + quota (per day/month)

MODULE 2.3: Caching (1 hour)

Setup

```
# Open Redis Commander for visual demo
start http://localhost:8091
```

Demo 1: Cache Miss vs Cache Hit (20 min)

```
# Get a payment ID first
PAYMENT_ID=$(curl -s -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: cache-demo-001" \
  -d '{"orderId":"ORDER-C1","customerId":"CUST-001","amount":"9.99","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","i
demo-001"}' \
  | python -c "import sys,json; print(json.load(sys.stdin)['paymentId'])")

echo "Payment ID: $PAYMENT_ID"

# First GET - CACHE MISS (hits database)
time curl http://localhost:8081/api/v2/payments/$PAYMENT_ID

# Watch logs: "[CACHE MISS] Loading payment from database"
# Note: DB query visible in H2 console logs

# Second GET - CACHE HIT (from Redis)
time curl http://localhost:8081/api/v2/payments/$PAYMENT_ID

# Watch logs: NO "[CACHE MISS]" log - served from Redis!
# Compare timing: ~150ms (DB) vs ~5ms (Redis)
```

Check Redis Commander:

1. Open <http://localhost:8091>
2. Navigate to `payments:payments:::{PAYMENT_ID}` key
3. Show the JSON value stored in Redis
4. Show TTL countdown (5 minutes)

Demo 2: Cache Eviction on Update (15 min)

```
# Cancel payment (triggers @CacheEvict)
curl -X DELETE http://localhost:8081/api/v1/payments/$PAYMENT_ID

# Watch logs: "Cache eviction triggered for paymentId"
# Check Redis Commander: key is GONE

# Next GET - CACHE MISS again (evicted)
curl http://localhost:8081/api/v2/payments/$PAYMENT_ID
# Watch logs: "[CACHE MISS] Loading payment from database"
```

Demo 3: HTTP Cache vs Spring Cache (15 min)

```
# Show Cache-Control headers on payment GET
curl -v http://localhost:8081/api/v2/payments/$PAYMENT_ID 2>&1 | grep "Cache-Control"

# Show CDN serving static content
```

```
curl -I http://localhost:8084/ui/index.html | grep -i cache
# Cache-Control: public, max-age=86400
# X-Cache-Status: HIT-CDN
```

Discussion comparison:

	Spring Cache (@Cacheable)	HTTP Cache (Cache-Control)
Where	Server-side (Redis)	Client/CDN/Proxy
Invalidation	@CacheEvict (immediate)	Wait for TTL or ETag
Visibility	Invisible to client	Client sees headers
Use for	DB results, expensive computations	Static content, public APIs

MODULE 2.4: Gateway & Discovery (1 hour)

Setup

```
# Start discovery-service
cd discovery-service && mvn spring-boot:run &

# Wait for Eureka to start (30 seconds)
sleep 30

# Start payment-service (registers with Eureka)
cd payment-service && mvn spring-boot:run &

# Start order-service (registers with Eureka)
cd order-service && mvn spring-boot:run &

# Start bff-service
cd bff-service && mvn spring-boot:run &

# Finally, start gateway
cd gateway-service && mvn spring-boot:run &

# Open Eureka dashboard
start http://localhost:8761
```

Demo 1: Service Discovery (20 min)

Show Eureka dashboard:

1. Navigate to <http://localhost:8761>
2. Point out: payment-service, order-service, bff-service registered
3. Show: instance info (IP, port, health check URL)
4. Show: lease info (last renewal, expiry)

```
# Query Eureka REST API directly
curl http://eureka:eureka-secret@localhost:8761/eureka/apps/payment-service \
```

```
| python -m json.tool
```

```
# Show: instanceInfo with hostName, port, status=UP
```

Scale payment-service (blue-green demo):

```
# Start second instance on port 8082
SERVER_PORT=8082 mvn spring-boot:run -pl payment-service &

# Wait 30 seconds -> check Eureka
# Show: payment-service shows 2 instances!
start http://localhost:8761
```

Demo 2: Gateway Routing (25 min)

```
# All traffic via gateway (port 8080)
# NOT directly to services (8081, 8082)

# Show gateway routes
curl http://localhost:8080/actuator/gateway/routes | python -m json.tool

# Test routing:
# Payment (route: /api/v1/payments -> lb://payment-service)
curl -H "X-API-Key: demo-key" \
  http://localhost:8080/api/v1/payments?customerId=CUST-001

# Order (route: /api/v1/orders -> lb://order-service)
curl -H "X-API-Key: demo-key" \
  http://localhost:8080/api/v1/orders?customerId=CUST-001

# BFF (route: /bff/** -> lb://bff-service)
curl -H "X-API-Key: demo-key" \
  "http://localhost:8080/bff/payment-summary/dashboard?customerId=CUST-001"
```

Show correlation ID propagation:

```
curl -v http://localhost:8080/api/v1/payments?customerId=CUST-001 \
  -H "X-API-Key: demo-key" \
  -H "X-Correlation-ID: my-trace-12345" 2>&1 | grep -i correlation

# Response shows: X-Correlation-ID: my-trace-12345
# Gateway added it to request -> payment-service logs show it too
```

Show auth validation:

```
# Without API key -> 401
curl http://localhost:8080/api/v1/payments?customerId=CUST-001
# Expected: 401 Unauthorized (from gateway auth filter)
```

```
# With API key -> routed to service
curl -H "X-API-Key: demo-key" \
  http://localhost:8080/api/v1/payments?customerId=CUST-001
# Expected: 200 OK
```

Demo 3: Blue-Green Routing (15 min)

```
# Canary route: X-Canary: true header routes to "green" deployment
curl -H "X-API-Key: demo-key" \
  -H "X-Canary: true" \
  http://localhost:8080/api/v1/payments?customerId=CUST-001

# Check response header: X-Canary-Routed: true
# In real setup: "green" = new version deployment
```

DAY 3 PLAYBOOK

Theme: Distributed Transactions, Saga, Observability, Design Patterns

MODULE 3.1: Distributed Transactions (1 hour)

Setup

```
# All services should be running
# Open Kafka UI
start http://localhost:8090
```

Demo 1: 2PC Problems Discussion (15 min)

Whiteboard drawing:

```
2PC PROBLEM:
Coordinator asks: "Can you commit?" to payment-service
payment-service: "YES" -> LOCKS the payment record
Coordinator asks: "Can you commit?" to order-service
order-service: "YES" -> LOCKS the order record

Coordinator FAILS before sending "COMMIT"

RESULT: Both services locked forever!
Payment can't be processed, order can't be updated.
System HANGS until coordinator recovers.

This is why 2PC is avoided in microservices!
```

Demo 2: Saga Happy Path (30 min)

```
# Step 1: Create order
ORDER_RESPONSE=$(curl -s -X POST http://localhost:8082/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUST-001","amount":"9.99","currency":"USD","description":"Training Demo Order"}')

echo $ORDER_RESPONSE | python -m json.tool
ORDER_ID=$(echo $ORDER_RESPONSE | python -c "import sys,json; print(json.load(sys.stdin)
['orderId'])")
echo "Order ID: $ORDER_ID | Status: CREATED"

# Step 2: Create payment (triggers Saga)
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: saga-demo-001" \
  -d '{"orderId":"$ORDER_ID","customerId":"CUST-001","amount":"9.99","currency":"USD","paymentMethod":"CARD","cardNumber":"4111 demo-001"}'

# Step 3: Watch Kafka events (in Kafka UI)
# Navigate to: http://localhost:8090 -> Topics -> payment-events
# See: PAYMENT_INITIATED -> PAYMENT_COMPLETED

# Step 4: Check order status updated by Saga
sleep 3
curl http://localhost:8082/api/v1/orders/$ORDER_ID | python -m json.tool
# Status: AWAITING_PAYMENT -> CONFIRMED -> COMPLETED
```

Point out in logs:

```
[payment-service] [EVENT] Published PAYMENT_COMPLETED for payment -> Kafka
[order-service] [SAGA] Received payment event: PAYMENT_COMPLETED for order
[order-service] [SAGA] Order ORD-xxx CONFIRMED after payment completed
```

Demo 3: Saga Compensation (15 min)

```
# Create order
ORDER_ID=$(curl -s -X POST http://localhost:8082/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUST-001","amount":"75.00","currency":"USD","description":"Will Fail - Insufficient Funds"}' \
  | python -c "import sys,json; print(json.load(sys.stdin)['orderId'])")

# Create payment with amount that FAILS (50-100 = insufficient funds)
curl -X POST http://localhost:8081/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Idempotency-Key: saga-comp-001" \
  -d '{"orderId":"$ORDER_ID","customerId":"CUST-001","amount":"75.00","currency":"USD","paymentMethod":"CARD","cardNumber":"4111 comp-001"}'
```



```
sleep 2
# Check order - should be CANCELLED (compensation executed)
curl http://localhost:8082/api/v1/orders/$ORDER_ID
# Status: CANCELLED (Saga compensation: payment failed -> order cancelled)
```

MODULE 3.2: Saga Styles - Orchestration vs Choreography (1 hour)

Demo 1: Choreography (implemented - show event flow) (20 min)

```
# Show the event-driven flow in Kafka UI
start http://localhost:8090
# Navigate: Topics -> payment-events
# Point out: No central coordinator
# Each service reacts to events independently

# payment-service publishes PAYMENT_COMPLETED
# order-service LISTENS and updates itself
# If order fails, order-service publishes ORDER_FAILED
# payment-service LISTENS and issues refund
```

Show PaymentEventConsumer.java:

```
// Highlight:
@KafkaListener(topics = "payment-events", groupId = "order-service")
public void handlePaymentEvent(PaymentEvent event, ...) {
    switch (event.getEventType()) {
        case PAYMENT_COMPLETED -> handlePaymentCompleted(event);
        case PAYMENT_FAILED    -> handlePaymentFailed(event); // compensation!
        case PAYMENT_REFUNDED  -> handlePaymentRefunded(event);
    }
}
```

Demo 2: Orchestration Discussion (20 min)

Draw on whiteboard:

ORCHESTRATION SAGA (not implemented, discussed):

```
[Saga Orchestrator] (central coordinator)
|
| 1. "InitiatePayment" command
▼
[Payment Service]
| "PaymentCompleted" reply
▼
[Saga Orchestrator]
| 2. "ConfirmOrder" command
▼
[Order Service]
```



Compare:

	Choreography	Orchestration
Coordinator	None (events)	Central orchestrator
Coupling	Loose	Tighter
Visibility	Hard to trace	Easy (one place)
SPOF	No	Orchestrator is SPOF
Testing	Complex	Easier
Used in training	✅ YES	Referenced

Demo 3: Timeout Handling (20 min)

```
# Show the scheduled timeout handler
# OrderService.java - @Scheduled(fixedDelay = 60000)

# Create an order but DON'T create payment
ORDER_ID=$(curl -s -X POST http://localhost:8082/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUST-001","amount":"9.99","currency":"USD","description":"Will Timeout"}' \
  | python -c "import sys,json; print(json.load(sys.stdin)['orderId'])")

echo "Order $ORDER_ID created. Waiting for timeout (5 min in config)..."
# In training: reduce to 1 minute to demo faster

# After timeout period:
curl http://localhost:8082/api/v1/orders/$ORDER_ID
# Status: CANCELLED (timeout compensation)
```

 MODULE 3.3: Distributed Tracing (1 hour)

Setup

```
# Open Zipkin
start http://localhost:9411

# Open Grafana
start http://localhost:3000 # admin/admin
```

Demo 1: Trace a Payment Across Services (30 min)

```
# Create a payment and capture trace ID
RESPONSE=$(curl -v -X POST http://localhost:8080/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-API-Key: demo-key" \
  -H "X-Idempotency-Key: trace-demo-001" \
  -d '{"orderId":"ORDER-T1","customerId":"CUST-001","amount":"9.99","currency":"USD","paymentMethod":"CARD","cardNumber":"4111111111111111","i
demo-001"}' 2>&1)

TRACE_ID=$(echo "$RESPONSE" | grep -i "x-trace-id" | awk '{print $3}' | tr -d '\r')
echo "Trace ID: $TRACE_ID"
```

Navigate Zipkin:

1. Open <http://localhost:9411>
2. Click "Search" -> Limit: 10 -> Run Query
3. Find trace with your traceId
4. Click on trace -> See all spans:
 - gateway-service span (routing)
 - payment-service span (business logic)
 - PSP call span (external call)
5. Point out: Timeline shows each service duration
6. Find slow PSP call (if amount >= 100)

Show traceId in logs:

```
# Payment-service logs show traceId in EVERY line
# [payment-service,abc123traceId,span456] PaymentService - Payment created
```

Demo 2: Golden Signals in Prometheus/Grafana (20 min)

```
# Prometheus queries to show:
open http://localhost:9090

# TRAFFIC: Requests per second
# Query: rate(payment_requests_total[5m])

# ERRORS: Error rate
# Query: rate(payment_failure_total[5m]) / rate(payment_requests_total[5m])

# LATENCY: P99 latency
# Query: histogram_quantile(0.99, payment_processing_duration_seconds_bucket)

# SATURATION: Active threads
# Query: jvm_threads_live_threads{application="payment-service"}
```

Grafana Dashboard:

1. Open <http://localhost:3000> (admin/admin)
2. Navigate to Dashboards -> Payment Service
3. Show golden signals dashboard
4. Point out: Circuit breaker gauge (0=CLOSED, 1=OPEN, 2=HALF_OPEN)

Demo 3: Correlation ID End-to-End (10 min)

```
# Send request with custom correlation ID
curl -H "X-API-Key: demo-key" \
      -H "X-Correlation-ID: my-business-tx-12345" \
      http://localhost:8080/api/v1/payments?customerId=CUST-001

# Check logs across ALL services for "my-business-tx-12345"
docker-compose logs payment-service | grep "my-business-tx-12345"
docker-compose logs order-service   | grep "my-business-tx-12345"
```

MODULE 3.4: Design Patterns (1 hour)

Demo 1: Patterns Overview Endpoint (20 min)

```
# Get all patterns with payment domain examples
curl http://localhost:8081/api/patterns/overview | python -m json.tool
```

Walk through each pattern in the response:

1. BFF - show bff-service aggregating
2. API Gateway - show gateway routes
3. Strangler Fig - show v1/v2 coexistence
4. ACL - show PspClient translating PSP model
5. CQRS - show write (POST) vs read (GET with Redis)
6. Event Sourcing - show Kafka events as event log
7. Saga - reference Day 3.1-3.2

Demo 2: BFF Pattern Live Demo (20 min)

```
# WITHOUT BFF: Client makes 3 calls
# 1. Payments
curl http://localhost:8081/api/v2/payments?customerId=CUST-001
# 2. Orders
curl http://localhost:8082/api/v1/orders?customerId=CUST-001
# 3. Need to combine manually

# WITH BFF: ONE call returns combined dashboard
curl "http://localhost:8080/bff/payment-summary/dashboard?customerId=CUST-001" \
      -H "X-API-Key: demo-key" | python -m json.tool

# Point out:
# - summary (computed stats)
# - recentPayments (from payment-service)
# - activeOrder (from order-service)
```

```
# - generatedAt timestamp
# BFF made PARALLEL calls internally!
```

Demo 3: Pattern Selection Guide (20 min)

```
# Show the guide
curl http://localhost:8081/api/patterns/guide | python -m json.tool
```

Interactive Q&A with trainees:

- "Mobile app is over-fetching data from payments API" -> BFF
- "Payment service down affects order service" -> Circuit Breaker
- "Need to replace monolith slowly" -> Strangler Fig
- "PSP API changes break our code" -> Anti-Corruption Layer
- "Need to audit every payment state change" -> Event Sourcing
- "Payment and order must stay consistent" -> Saga

DAY 4 PLAYBOOK

Theme: Edge, DNS, Load Balancer, WAF, CDN

MODULE 4.1: DNS & Traffic (1 hour)

Demo 1: DNS Flow Explanation (30 min)

```
# Get the full DNS flow explanation
curl http://localhost:8081/api/edge/traffic-flow | python -m json.tool
curl http://localhost:8081/api/edge/dns-flow | python -m json.tool
```

Windows local DNS demo:

```
# As Administrator in Windows Terminal

# Step 1: Open hosts file
notepad C:\Windows\System32\drivers\etc\hosts

# Add line at bottom:
# 127.0.0.1 api.payments.local

# Step 2: Verify DNS resolution
nslookup api.payments.local
# Expected: 127.0.0.1

# Step 3: Call API via local domain
curl -H "X-API-Key: demo-key" "http://api.payments.local:8080/api/v1/payments?
customerId=CUST-001"

# Step 4: Show DNS cache
ipconfig /displaydns | findstr /I payments
```

```
# Step 5: Flush DNS cache (simulate TTL expiry)
ipconfig /flushdns
```

Discussion: CNAME for CDN:

```
api.payments.example.com    CNAME  d1234.cloudfront.net
d1234.cloudfront.net       A       13.225.100.50  (US East CDN)
                           A       52.85.200.30   (EU CDN, if geo-routed)
```

WHY CNAME?

- CDN provider manages the A records (IPs change as they scale)
- You only manage your CNAME (pointing to CDN)
- CDN geo-routes based on client location automatically

Demo 2: Geo Routing Simulation (30 min)

```
# Show geo routing concept from API
curl http://localhost:8081/api/edge/dns-flow | python -m json.tool | grep -A 10 "geoRouting"

# Demo: Different DNS responses based on location
# (Simulated in the response - actual geo routing requires real DNS)

# Discuss TTL impact
# Low TTL = fast failover
# High TTL = cached longer, fewer DNS queries

# Practical demo: Change hosts file to wrong IP
# Edit hosts: 127.0.0.1 -> 192.168.1.99
# curl api.payments.local -> fails
# Change back -> ipconfig /flushdns -> works again
# This shows TTL impact on DNS changes
```

MODULE 4.2: Load Balancer (1 hour)

Demo 1: L4 vs L7 Load Balancer (25 min)

```
# Show L4 vs L7 explanation
curl http://localhost:8081/api/edge/traffic-flow | python -m json.tool | grep -A 30
"hop4_load_balancer"
```

Demonstration using Eureka as L7 LB:

```
# Start two payment-service instances
mvn spring-boot:run -pl payment-service &
SERVER_PORT=8082 mvn spring-boot:run -pl payment-service &

# Both register in Eureka
start http://localhost:8761
```

```
# Show: PAYMENT-SERVICE has 2 instances

# Gateway round-robins between them
# Send 10 requests, watch which instance handles each
for i in {1..10}; do
    curl -s -H "X-API-Key: demo-key" \
        http://localhost:8080/api/v1/payments?customerId=CUST-001 | \
        python -c "import sys,json; pass" &
done
wait

# Watch logs of each instance: both handle requests!
```

Demo 2: Health Checks & Instance Removal (20 min)

```
# Show health check endpoint
curl http://localhost:8081/actuator/health | python -m json.tool

# Kill payment-service (simulate crash)
# Ctrl+C on payment-service terminal

# Wait 30 seconds (Eureka detects failure)
sleep 30

# Check Eureka - down instance shown differently
start http://localhost:8761

# Send request via gateway - still works! (routes to healthy instance)
curl -H "X-API-Key: demo-key" \
    http://localhost:8080/api/v1/payments?customerId=CUST-001
# 200 OK - gateway fails over to healthy instance

# Restart payment-service
mvn spring-boot:run -pl payment-service &
# Within 30 seconds: re-registers, traffic flows again
```

Demo 3: TLS Termination Discussion (15 min)

```
# Show traffic flow from API
curl http://localhost:8081/api/edge/traffic-flow | python -m json.tool | \
    grep -A 10 "tlsTermination"
```

Whiteboard:

CLIENT (HTTPS) -> [Load Balancer] -> BACKEND (HTTP)
TLS Terminates
here

Benefits:

- Backend services don't need TLS certs

- Simpler configuration for services
- LB handles TLS overhead (offload)
- Internal network is VPC/private (trusted)

In training: HTTP everywhere (no TLS for simplicity)

In production: HTTPS to LB, HTTP internally within VPC

MODULE 4.3: WAF & DDoS (1 hour)

Demo 1: WAF Request Inspection (30 min)

```
# Clean request - ALLOWED
curl -X POST http://localhost:8081/api/edge/waf-check \
  -H "Content-Type: application/json" \
  -d '{"customerId": "CUST-001", "amount": "9.99"}'
# Expected: decision=ALLOW

# SQL Injection - BLOCKED
curl -X POST http://localhost:8081/api/edge/waf-check \
  -H "Content-Type: application/json" \
  -d '{"customerId": "1\'\''; DROP TABLE payments;--"}'
# Expected: decision=BLOCK, blockReason=SQL_INJECTION_DETECTED

# XSS Attack - BLOCKED
curl -X POST http://localhost:8081/api/edge/waf-check \
  -H "Content-Type: application/json" \
  -d '{"description": "<script>alert(\"xss\")</script>"}'
# Expected: decision=BLOCK, blockReason=XSS_DETECTED

# Path Traversal - BLOCKED
curl -X POST http://localhost:8081/api/edge/waf-check \
  -H "Content-Type: application/json" \
  -d '{"path": "../../../etc/passwd"}'
# Expected: decision=BLOCK, blockReason=PATH_TRAVERSAL_DETECTED

# Bot User-Agent - WARNING
curl -X POST http://localhost:8081/api/edge/waf-check \
  -H "Content-Type: application/json" \
  -H "User-Agent: sqlmap/1.0" \
  -d '{"customerId": "CUST-001"}'
# Expected: decision=ALLOW with warnings (scanner detected)

# Blocked IP - BLOCKED
curl -X POST http://localhost:8081/api/edge/waf-check \
  -H "Content-Type: application/json" \
  -H "X-Forwarded-For: 10.0.0.1" \
  -d '{"customerId": "CUST-001"}'
# Expected: decision=BLOCK, blockReason=IP_BLOCKLIST
```

Demo 2: OWASP Top 10 Discussion (20 min)


```
# Get full WAF explanation
curl http://localhost:8081/api/edge/traffic-flow | python -m json.tool | grep -A 50
"hop3_waf"
```

Walk through OWASP threats relevant to payment APIs:

1. **A01 Broken Access Control:** Customer viewing other customer's payments
 - Demo: Try `customerId=ADMIN` - should be rejected
2. **A03 Injection:** SQL injection in payment fields
 - Demo: SQL injection demo above
3. **A07 Auth Failures:** Brute force API keys
 - Demo: Rate limiting blocks this at gateway
4. **A09 Logging Failures:** No audit trail
 - Demo: Zipkin shows all payment traces with `traceld`

Demo 3: Rate Limiting as DDoS Protection (10 min)

```
# DDoS simulation - hammer the API
for i in {1..100}; do
  curl -s -o /dev/null -w "%{http_code}" \
    -H "X-API-Key: ddos-test" \
    http://localhost:8081/api/v1/payments?customerId=CUST-001
  echo " request $i"
done
# 200s -> 429s (rate limited)
# Without gateway: service would be overwhelmed
# With gateway rate limiting: excess requests blocked at edge
```

MODULE 4.4: CDN & Static Content (1 hour)

Demo 1: NGINX as CDN Simulation (25 min)

```
# Show NGINX CDN serving static files
curl -I http://localhost:8084/ui/index.html

# Expected headers:
# Cache-Control: public, max-age=86400, immutable
# X-Cache-Status: HIT-CDN
# X-Served-By: nginx-cdn-simulation
# Content-Type: text/html

# Open in browser
start http://localhost:8084/ui/index.html
```

Compare response times:

```
# Static from CDN (NGINX)
time curl http://localhost:8084/ui/index.html -o /dev/null
```

```
# Dynamic from service
time curl http://localhost:8081/api/v2/payments?customerId=CUST-001

# CDN: ~2-5ms | Service: ~100-300ms
# CDN is 20-60x faster!
```

Show cache miss vs hit:

```
# API request (no cache)
curl -I http://localhost:8084/api/v1/payments?customerId=CUST-001
# X-Cache-Status: BYPASS (API requests not cached!)
# Cache-Control: no-store, no-cache

# Static request (cached)
curl -I http://localhost:8084/ui/index.html
# Cache-Control: public, max-age=86400
```

Demo 2: CDN Strategy for Payments (20 min)

```
# Get CDN strategy explanation
curl http://localhost:8081/api/edge/cdn-strategy | python -m json.tool
```

Key Points Discussion:

1. **Cache-Control: no-store** for all payment creation/status APIs
 - Payment data is user-specific, sensitive, dynamic
2. **Cache-Control: public, max-age=86400** for UI assets
 - JavaScript, CSS, images: immutable, same for all users
3. **OpenAPI docs** can be CDN cached (changes rarely)
4. **COMPLETED status** - borderline cacheable with short TTL

Demo 3: Full Request Journey Summary (15 min)

```
# Show complete traffic flow one more time
curl http://localhost:8081/api/edge/traffic-flow?
domain=api.payments.example.com&path=/api/v1/payments | python -m json.tool
```

Walk through each hop with timing:

DNS:	1ms (cached) - 50ms (cold)
CDN Edge:	0ms (cache hit for static) - 50ms (miss, forward)
WAF:	1-5ms (rule matching)
Load Balancer:	1-3ms (routing decision)
API Gateway:	5-15ms (auth, rate limit, route)
Service:	100-500ms (DB, PSP, cache)
TOTAL:	108-623ms typical API call
	5-20ms for CDN-cached static content

TROUBLESHOOTING GUIDE

Service Won't Start

```
# Check port conflicts
netstat -an | findstr "8081"

# Check Maven build
mvn clean install -DskipTests -pl payment-service -am

# Check logs
tail -100 payment-service/target/spring.log
```

Kafka Not Receiving Events

```
# Check Kafka is running
docker exec payments-kafka kafka-broker-api-versions --bootstrap-server localhost:9092

# Create topics manually if needed
docker exec payments-kafka kafka-topics --create \
  --bootstrap-server localhost:9092 \
  --topic payment-events \
  --partitions 3 \
  --replication-factor 1
```

Redis Connection Failed

```
# Check Redis
docker exec payments-redis redis-cli ping

# If NOAUTH error - check redis.conf has no requirepass
docker exec payments-redis redis-cli config get requirepass
```

Eureka Not Registering Services

```
# Check discovery service is UP first
curl http://localhost:8761/actuator/health

# Check service Eureka URL in application.yml
# eureka.client.service-url.defaultZone must match discovery-service URL
```

Circuit Breaker Not Opening

```
# Reduce thresholds for demo in application.yml:
# sliding-window-size: 5 (instead of 10)
```

```
# failure-rate-threshold: 40 (instead of 50)
# Then trigger failures with amount $50-$99.99
```

QUICK REFERENCE - ALL DEMO URLs

Tool	URL	Credentials
Payment API Swagger	http://localhost:8081/swagger-ui.html	X-API-Key: demo-key
Order API Swagger	http://localhost:8082/swagger-ui.html	-
BFF API Swagger	http://localhost:8083/swagger-ui.html	-
Eureka Dashboard	http://localhost:8761	eureka/eureka-secret
Zipkin Traces	http://localhost:9411	-
Prometheus	http://localhost:9090	-
Grafana	http://localhost:3000	admin/admin
Kafka UI	http://localhost:8090	-
Redis Commander	http://localhost:8091	-
CDN Static	http://localhost:8084/ui/index.html	-
H2 Console (Payment)	http://localhost:8081/h2-console	sa/[empty]
H2 Console (Order)	http://localhost:8082/h2-console	sa/[empty]
Circuit Breakers	http://localhost:8081/actuator/circuitbreakers	-
Gateway Routes	http://localhost:8080/actuator/gateway/routes	-
Patterns Overview	http://localhost:8081/api/patterns/overview	-
WAF Demo	http://localhost:8081/api/edge/waf-check	-
Traffic Flow	http://localhost:8081/api/edge/traffic-flow	-
DNS Flow	http://localhost:8081/api/edge/dns-flow	-
CDN Strategy	http://localhost:8081/api/edge/cdn-strategy	-

KEY LEARNING OUTCOMES CHECKLIST

Day 1

- ☐ Trainees can explain REST resource naming
- ☐ Trainees can describe offset vs cursor pagination tradeoffs
- ☐ Trainees understand idempotency key pattern for payments
- ☐ Trainees can read and write OpenAPI annotations
- ☐ Trainees understand multi-module Maven structure

- ☐ Trainees can explain cyclic dependency problem

Day 2

- ☐ Trainees can draw circuit breaker state machine
- ☐ Trainees explain token bucket algorithm
- ☐ Trainees understand @Cacheable/@CacheEvict annotation behavior
- ☐ Trainees can explain Spring Cache vs HTTP Cache
- ☐ Trainees understand Eureka registration flow
- ☐ Trainees can trace a request through gateway

Day 3

- ☐ Trainees can explain 2PC vs Saga tradeoffs
- ☐ Trainees can draw choreography vs orchestration flow
- ☐ Trainees understand eventual consistency
- ☐ Trainees can find a trace in Zipkin by traceId
- ☐ Trainees know the 4 golden signals
- ☐ Trainees can map each pattern to a payment problem

Day 4

- ☐ Trainees can trace DNS lookup steps
- ☐ Trainees explain L4 vs L7 load balancer
- ☐ Trainees identify which content to CDN cache
- ☐ Trainees can name 3 WAF-blocked threat types
- ☐ Trainees explain TLS termination at LB
- ☐ Trainees understand geo-routing via DNS

End of Trainer Playbook